

Proyecto integrador IA2 — LogiTrack IQ: Torre de control de inventario

Descripción del reto

LogiTrack S.A. ya tiene un backend en Spring Boot para bodegas, productos y movimientos de inventario. La información existe, pero se revisa manualmente y no hay una vista diaria que ayude a detectar faltantes ni a preparar una compra.

En este proyecto cada estudiante extenderá ese backend para construir una torre de control sencilla.

El sistema identifica un producto en riesgo, prepara una orden de compra, registra su recepción y muestra el resultado en un dashboard. Un flujo de n8n consulta el sistema mediante MCP y publica un resumen diario estructurado.

Este proyecto **extiende** el reto anterior de LogiTrack. No se crea un backend independiente ni se reemplazan sus funciones ya construidas.

Objetivo general

Integrar Spring Boot, pruebas, SDD, MCP, skills y n8n en una solución pequeña que monitoree inventario, proponga una compra en borrador y entregue información clara a un administrador.

¿Qué sistema se debe construir?

Se debe construir una extensión de LogiTrack que complete este flujo de negocio:

1. El sistema calcula el inventario real a partir de los movimientos registrados.
2. Detecta productos cuyo stock está por debajo de su punto de reorden.
3. El flujo diario de n8n consulta esa información mediante MCP y crea, como máximo, una orden de compra en estado **BORRADOR**.
4. Un administrador revisa la orden en el dashboard y la aprueba.
5. Cuando la orden se recibe, el backend registra automáticamente una entrada de inventario en la bodega indicada.
6. El dashboard muestra los indicadores, las alertas, las órdenes pendientes y el inventario actualizado.

En pocas palabras: **el sistema detecta un faltante, prepara una compra, permite recibirla y demuestra que el inventario se actualizó.**

¿Qué se debe lograr al finalizar?

Al terminar, el estudiante debe poder demostrar, con datos reales del sistema, un flujo completo de principio a fin:

- existe un producto en riesgo;

- n8n consulta el backend mediante MCP y crea una orden en BORRADOR ;
- un ADMIN aprueba y recibe esa orden;
- la recepción crea un movimiento ENTRADA ;
- el dashboard refleja la orden y el inventario actualizado.

Si este flujo funciona, está probado y se evidencia en el video, el objetivo principal del proyecto está cumplido.

Glosario breve

Término	Significado en este proyecto
Orden de compra	Registro del sistema que propone comprar un producto para una bodega. No es un PDF por sí misma.
BORRADOR	Estado inicial de una orden. La orden existe en la base de datos, pero todavía no ha sido aprobada ni recibida.
PDF de la orden	Documento generado desde una orden guardada. Si la orden está en BORRADOR , lleva la marca de agua diagonal BORRADOR.
Punto de reorden	Nivel de inventario que permite decidir si un producto debe aparecer en riesgo. Su fórmula está definida en este documento.
MCP	Capa que permite que el flujo de n8n consulte o use funciones limitadas del backend mediante herramientas.
Skill	Archivo de instrucciones operativas que indica qué puede y qué no puede hacer el flujo automatizado.
Resumen del panel	Contenido estructurado que el flujo publica para que el dashboard muestre narrativa, alertas y acciones.

Convenciones que aplican a todo el proyecto

- **Fuente de verdad:** el backend y su base de datos son la única fuente de información. El dashboard, MCP y n8n consultan o usan la API; no calculan ni modifican datos directamente en MySQL.
- **Estados y permisos:** una orden solo cambia con las transiciones y roles definidos más adelante. Que una acción aparezca en el dashboard no significa que todos los usuarios puedan ejecutarla.
- **Errores:** se reutiliza el manejo global de excepciones del backend anterior. Las validaciones y transiciones inválidas usan 400 ; recursos inexistentes usan 404 ; acciones prohibidas por rol usan 403 ; una sesión no válida usa 401 .
- **Evidencia funcional:** una captura, un DTO o un endpoint documentado no sustituyen una ejecución real. Cada requisito debe poder comprobarse desde la API, n8n, el dashboard, una prueba automatizada o el video, según corresponda.
- **Alcance:** cuando el documento dice “debe”, es obligatorio y se califica. Cuando dice “puede”, es una alternativa de implementación que no cambia el comportamiento esperado.

Reglas base de inventario

Estas reglas eliminan ambigüedades del proyecto anterior.

- La zona horaria de backend, n8n y datos de prueba es `America/Bogota`.
- La capacidad de una bodega se mide en unidades de producto y debe ser mayor que 0.
- El stock se calcula a partir de los movimientos; el campo `Producto.stock` del reto anterior no es fuente para estos cálculos nuevos.
- Un movimiento puede tener uno o varios detalles de producto. Cada cálculo debe recorrer sus detalles. Si la implementación anterior guarda un solo producto por movimiento, se aplica la misma regla a ese producto.
- Al iniciar las pruebas, los datos deben incluir movimientos `ENTRADA` que representen inventario inicial.
- `ENTRADA` suma unidades a la bodega destino; `SALIDA` resta unidades a la bodega origen; `TRANSFERENCIA` resta en origen y suma la misma cantidad en destino.
- No se permite una salida o transferencia que deje una bodega con stock negativo.
- El stock total de un producto es la suma de sus existencias en todas las bodegas.

Alcance funcional obligatorio

1. Modelo mínimo

Agregar o adaptar estas entidades al backend existente:

- **Proveedor:** `id`, `nombre`, `contacto`, `diasEntrega`.
 - Los proveedores se cargan con `data.sql` o con un mecanismo equivalente y reproducible.
 - `diasEntrega` es un entero entre 1 y 90.
- **Producto:** agregar la relación opcional `proveedorPrincipal` (`ManyToOne` con `Proveedor`).
 - Un producto sin proveedor principal no puede aparecer como producto en riesgo ni generar una orden automática.
- **OrdenCompra:** `id`, `producto`, `proveedor`, `bodegaDestino`, `cantidad`, `precioUnitario`, `total`, `fechaCreacion`, `estado`, `creadoPor`.
 - Tiene exactamente un producto y una cantidad mayor que 0.
 - `bodegaDestino` es obligatoria.
 - Debe almacenar opcionalmente el PDF generado de la orden y su fecha de generación.
- **ResumenPanel:** `id`, `fecha`, `contenidoJson`, `autor`.
 - Solo puede haber un resumen válido por fecha. Una nueva publicación para la misma fecha reemplaza el contenido anterior y queda registrada en auditoría.

2. Indicadores y criterios fijos

El dashboard muestra estos cuatro indicadores:

Indicador	Regla exacta
Ocupación por bodega	$(\text{unidades almacenadas en la bodega} / \text{capacidad}) \times 100$. Se muestra por cada bodega.
Productos en quiebre	Cantidad de productos cuyo stock total es <code>0</code> .
Productos en riesgo	Cantidad de productos con proveedor principal cuyo stock total es menor que su punto de reorden.
Órdenes por aprobar	Cantidad de órdenes en <code>BORRADOR</code> y suma de sus totales.

Para cada producto en riesgo también se calculan estos datos, que se muestran en su tabla:

- **Consumo diario promedio:** unidades en movimientos `SALIDA` de los últimos 30 días calendario, incluida la fecha de consulta, dividido entre `30` .
- **Punto de reorden:** $\text{consumo diario promedio} \times \text{diasEntrega} \times 1.5$.
- **Días de cobertura:** $\text{stock total} / \text{consumo diario promedio}$.
 - Si el consumo es `0` , el valor es `null` y el estado mostrado es `SIN_CONSUMO` .
 - Si el stock es igual al punto de reorden, el producto **no** está en riesgo; debe ser menor.
- **Movimientos de ayer:** conteo separado de `ENTRADA` , `SALIDA` y `TRANSFERENCIA` del día calendario anterior en `America/Bogota` . Se muestra como bloque informativo, no como tarjeta principal.

Una bodega es **crítica** cuando su ocupación es mayor o igual a `90 %` .

3. Estados de la orden y recepción

Los estados permitidos son `BORRADOR` , `APROBADA` , `RECIBIDA` y `CANCELADA` .

Estado actual	Siguiente estado permitido
<code>BORRADOR</code>	<code>APROBADA</code> o <code>CANCELADA</code>
<code>APROBADA</code>	<code>RECIBIDA</code> o <code>CANCELADA</code>
<code>RECIBIDA</code>	Ninguno
<code>CANCELADA</code>	Ninguno

Una transición no listada responde `400 Bad Request` con un mensaje claro.

Al pasar una orden de `APROBADA` a `RECIBIDA` , el sistema crea automáticamente un movimiento `ENTRADA` para su producto, cantidad y `bodegaDestino` . La actualización de la orden y la creación del movimiento deben ocurrir en una sola transacción: ambas se completan o ninguna se guarda.

4. API requerida

Se mantienen los endpoints del reto anterior. Agregar estos endpoints documentados en Swagger/OpenAPI:

Método y ruta	Respuesta o comportamiento mínimo
GET /kpis	Entrega los cuatro indicadores, los movimientos de ayer y calculadoEn .
GET /productos/{id}/stock	Entrega el stock total y el desglose por bodega, calculados desde movimientos.
GET /productos/riesgo	Lista productos en riesgo con proveedor, stock, consumo, punto de reorden, cobertura y bodega destino sugerida.
GET /bodegas/criticas	Lista bodegas con ocupación mayor o igual a 90 %.
GET /proveedores	Devuelve los proveedores precargados.
GET /ordenes	Devuelve las órdenes; acepta opcionalmente el filtro estado .
POST /ordenes	Crea una orden en BORRADOR ; calcula total en el servidor.
GET /ordenes/{id}	Devuelve una orden.
POST /ordenes/{id}/pdf	Genera y guarda el documento PDF de la orden.
GET /ordenes/{id}/pdf	Devuelve el PDF guardado para visualizarlo o descargarlo.
PATCH /ordenes/{id}/estado	Cambia el estado con las reglas definidas.
POST /panel/resumen	Valida y publica un resumen estructurado.
GET /panel/resumen	Devuelve el último resumen válido o 404 si no existe.

Ejemplo fijo de GET /kpis :

```
{
  "calculadoEn": "2026-08-24T06:00:00-05:00",
  "ocupacionPorBodega": [{ "bodegaId": 1, "nombre": "Bogota", "porcentaje": 92.5 }],
  "productosEnQuiebre": 1,
  "productosEnRiesgo": 2,
  "ordenesPorAprobar": { "cantidad": 1, "montoTotal": 45000.0 },
  "movimientosAyer": { "entrada": 2, "salida": 3, "transferencia": 1 }
}
```

GET /productos/riesgo debe incluir por cada elemento: productoId , nombreProducto , proveedorId , stockTotal , consumoDiarioPromedio , puntoReorden , diasCobertura , estadoCobertura y bodegaDestinoId .

La bodegaDestinoId sugerida es la bodega que tenga el menor stock de ese producto. Si hay empate, se usa la de menor id .

PATCH /ordenes/{id}/estado recibe exactamente un objeto como { "estado": "APROBADA" } .

Documento PDF de la orden

La orden debe poder generarse y guardarse como PDF dentro del sistema.

- `POST /ordenes/{id}/pdf` genera el PDF y lo guarda asociado a la orden. Si ya existe, lo reemplaza.
- El PDF debe incluir: número de orden, fecha de creación, proveedor, producto, cantidad, precio unitario, total, bodega destino y estado.
- Cuando la orden está en estado `BORRADOR`, el PDF debe mostrar una marca de agua diagonal, semitransparente y legible con el texto **BORRADOR**.
- `GET /ordenes/{id}/pdf` entrega el archivo con tipo `application/pdf` para visualizarlo en el navegador o descargarlo.
- Si aún no se ha generado el PDF, `GET /ordenes/{id}/pdf` responde `404`.
- Al cambiar el estado de una orden, el PDF guardado se elimina. El documento debe generarse nuevamente para reflejar el estado actual.

5. Contrato del resumen del panel

`POST /panel/resumen` acepta solo esta estructura y no admite propiedades adicionales:

```
{
  "fecha": "2026-08-24",
  "narrativa": "Hay productos en riesgo y una orden pendiente de aprobación.",
  "alertas": [
    {
      "severidad": "ALTA",
      "titulo": "Producto en riesgo",
      "detalle": "Producto X está por debajo de su punto de reorden.",
      "productoId": 12,
      "ordenId": null,
      "bodegaId": 3
    }
  ],
  "accionesSugeridas": [
    {
      "tipo": "REVISAR_ORDEN",
      "descripcion": "Revisar la orden 14 antes de aprobarla.",
      "ordenId": 14,
      "productoId": null,
      "bodegaId": null
    }
  ]
}
```

Reglas:

- `fecha` usa `YYYY-MM-DD` y corresponde a la fecha actual en `America/Bogota`.
- `narrativa` tiene entre 20 y 500 caracteres.
- `alertas` y `accionesSugeridas` son arreglos, aunque estén vacíos.
- `severidad`: `BAJA`, `MEDIA` o `ALTA`.
- `tipo`: `REVISAR_ORDEN`, `REVISAR_PRODUCTO` o `REVISAR_BODEGA`.
- Cada identificador informado debe existir.
- Una alerta enlaza al menos un identificador; una acción enlaza exactamente uno.
- Un JSON inválido responde `400` y el último resumen válido permanece disponible.

- Se valida estructura, longitud, enumeraciones y existencia de IDs. **No se exige validar el significado de la narrativa en lenguaje natural.**

Se permite implementar esta validación con DTOs, Bean Validation y comprobaciones de servicio; no se exige una librería externa de JSON Schema.

6. Seguridad y auditoría

Se reutilizan JWT, usuarios y auditoría del proyecto anterior. Agregar el rol `AGENTE` :

Acción	AGENTE	ADMIN
Consultar KPIs, stock, riesgos y bodegas críticas	Sí	Sí
Crear orden en <code>BORRADOR</code>	Sí	Sí
Publicar resumen	Sí	Sí
Aprobar, recibir o cancelar una orden	No	Sí
Registrar movimientos manualmente	No	Sí

La auditoría debe registrar las acciones que cambian el estado del sistema: creación de orden, publicación/reemplazo de resumen, transición de orden y recepción. No es obligatorio auditar consultas.

7. Servidor MCP

Crear un servidor MCP pequeño que llame a la API REST usando un usuario con rol `AGENTE` . No accede directamente a MySQL ni implementa reglas de negocio.

Debe tener exactamente seis herramientas:

1. `consultar_stock_producto(productoId)` → usa `GET /productos/{id}/stock` .
2. `consultar_bodegas_criticas()` → usa `GET /bodegas/criticas` .
3. `consultar_productos_en_riesgo()` → usa `GET /productos/riesgo` .
4. `consultar_kpis()` → usa `GET /kpis` .
5. `crear_orden_borrador(productoId, proveedorId, bodegaDestinoId, cantidad, precioUnitario)` → usa `POST /ordenes` .
6. `publicar_resumen(resumen)` → usa `POST /panel/resumen` .

No existe una herramienta para aprobar órdenes. Esa restricción es obligatoria.

8. Skill y flujo n8n

Crear `skills/operacion-logitrack/SKILL.md` . Debe indicar, como mínimo:

- consultar primero riesgos y KPIs;
- crear máximo una orden en borrador por ejecución;
- no aprobar, cancelar ni recibir órdenes;
- publicar solo un JSON que cumpla el contrato del resumen;
- informar el error si una herramienta falla.

Para el flujo, estas mismas reglas se copian o adaptan como instrucciones del nodo **AI Agent**. El archivo `SKILL.md` es la evidencia mantenible de esas instrucciones; no se exige cargarlo dinámicamente desde n8n.

Crear un único flujo n8n llamado **Resumen diario de inventario** :

1. Tiene un `Schedule Trigger` a las 6:00 a. m. en `America/Bogota` .
2. Tiene un nodo **AI Agent** que usa las herramientas MCP y sigue la skill creada.
3. Consulta KPIs y productos en riesgo.
4. Si encuentra productos en riesgo, crea como máximo una orden para el primer producto listado. La cantidad es `ceil(max(1, puntoReorden × 2 - stockTotal))` .
5. Publica el resumen del panel.
6. Registra una salida de éxito o error en la ejecución de n8n.

Para la demostración se permite ejecutar manualmente **ese mismo flujo**, sin cambiar su cronograma.

9. Dashboard web

En `frontend/` , crear HTML, CSS y JavaScript sin framework. Debe:

- mostrar los cuatro indicadores, movimientos de ayer y ocupación por bodega;
- mostrar la narrativa, alertas y acciones del último resumen;
- mostrar productos en riesgo y órdenes en `BORRADOR` ;
- permitir generar y visualizar el PDF de una orden en `BORRADOR` ; el documento debe mostrar la marca de agua diagonal **BORRADOR**;
- reutilizar el login JWT del proyecto anterior;
- guardar el JWT solo en `sessionStorage` ;
- mostrar el botón **Aprobar** solo a un `ADMIN` autenticado;
- actualizar la tabla después de aprobar una orden.

No se califican animaciones, interfaz móvil ni diseño avanzado. Sí se califica legibilidad y consumo de endpoints reales.

Pruebas obligatorias

Las pruebas de reglas nuevas se escriben antes de implementar esas reglas. Deben cubrir como mínimo:

1. consumo `0` : cobertura `null` y estado `SIN_CONSUMO` ;
2. stock igual al punto de reorden: no está en riesgo;
3. cantidad `0` o negativa: `400` ;
4. orden cancelada: no se puede aprobar (`400`);
5. orden aprobada recibida: genera una entrada;
6. `AGENTE` intenta aprobar: `403` ;
7. resumen con severidad inválida o ID inexistente: `400` y se conserva el resumen anterior;
8. PDF de una orden en `BORRADOR` : se guarda y contiene la marca de agua **BORRADOR**; al cambiar el estado, ya no queda disponible hasta generarlo nuevamente.

Agregar al menos una prueba de integración para PATCH /ordenes/{id}/estado o POST /panel/resumen .

Proceso SDD y evidencia TDD

Antes de implementar, crear docs/sdd/ con:

- 01-propuesta.md : problema, objetivo, alcance y fuera de alcance.
- 02-especificacion.md : reglas, contratos y ejemplo de resumen.
- 03-diseno.md : entidades, decisiones y diagrama del flujo.
- 04-tareas.md : tareas pequeñas con [] o [x] .

Crear además docs/sdd/evidencia-sdd.md , que incluya:

- enlaces relativos a los cuatro documentos;
- tabla regla → prueba;
- hash de estos tres commits, en este orden:
 - docs: define LogiTrack IQ scope
 - test: define reorder and order-state rules
 - feat: implement LogiTrack IQ rules
- evidencia de prueba inicial fallando y ejecución final en verde;
- reflexión de máximo 150 palabras sobre cambios entre especificación e implementación, o “No hubo cambios”.

Esta evidencia certifica de forma verificable el uso de SDD y TDD. No basta con afirmarlo en el README.

Estructura de referencia

```
src/                                # Mantiene la organización del backend anterior
frontend/
mcp-server/
n8n/
  resumen-diario-inventario.json
skills/
  operacion-logitrack/
    SKILL.md
docs/
  sdd/
README.md
schema.sql y data.sql, o instrucciones reproducibles equivalentes
```

La estructura exacta puede adaptarse al proyecto anterior, siempre que las responsabilidades estén separadas de forma clara.

Resultados esperados y entregables

1. **COMMITTS.** Se **DEBE** hacer un commit diario. El primer día se muestra la creación del repositorio y por lo menos este documento en una carpeta "docs" en la raíz del proyecto.

2. Repositorio Git, backend extendido y README con instalación, ejecución, usuarios de prueba y rutas principales.
3. Datos reproducibles: `schema.sql` y `data.sql` , o instrucciones equivalentes para cargar la base de datos.
4. Swagger/OpenAPI y evidencias de endpoints protegidos.
5. `docs/sdd/` completo, incluida `evidencia-sdd.md` con trazabilidad y hashes de commits.
6. `mcp-server/` con instrucciones y evidencia de entrada/respuesta para cada herramienta.
7. `skills/operacion-logitrack/SKILL.md` .
8. Export `n8n/resumen-diario-inventario.json` y captura de una ejecución exitosa y de un error controlado registrado.
9. `frontend/` conectado a la API real, con generación y visualización del PDF de una orden en `BORRADOR` .
10. Evidencia de un PDF guardado con la marca de agua diagonal **BORRADOR**.
11. Diagrama: n8n → MCP → API Spring Boot → MySQL → dashboard.
12. **Video de 4 a 6 minutos**. Debe mostrar este flujo:
 - datos iniciales y ejecución manual del flujo n8n;
 - consulta de riesgo y creación de una orden `BORRADOR` ;
 - aprobación por un `ADMIN` ;
 - recepción, movimiento `ENTRADA` y actualización del dashboard.

Puede mostrar n8n, Swagger, navegador, base de datos y MCP. **No debe mostrar código, archivos de código ni explicar código**. El video demuestra funcionamiento y decisiones del producto.

Rúbrica de calificación

Cada criterio tiene cinco niveles: **0, 30, 60, 80 y 100**. La nota final es el promedio simple de los cinco criterios.

1. Reglas, modelo y pruebas

- **0:** No hay modelo ni pruebas.
- **30:** Modelo incompleto y reglas principales fallan.
- **60:** Orden, cálculos o estados funcionan parcialmente.
- **80:** Modelo coherente, reglas y casos límite con pruebas funcionales.
- **100:** Todo lo anterior más pruebas escritas antes, recepción transaccional y evidencia roja/verde.

2. Flujo completo y video de presentación

- **0:** No hay video, el video muestra o explica código, o no se puede demostrar un flujo funcional.

- **30:** El video muestra pantallas aisladas, pero no demuestra que una orden pase por el flujo esperado.
- **60:** Se demuestra parte del flujo: existe un producto en riesgo y se crea o aprueba una orden, pero falta recibirla o comprobar el movimiento de entrada.
- **80:** El video demuestra el flujo completo: producto en riesgo, orden en **BORRADOR**, aprobación por **ADMIN**, recepción y movimiento **ENTRADA**.
- **100:** Todo lo anterior más dashboard actualizado al final, narración clara de la decisión de compra y video de 4 a 6 minutos sin mostrar ni explicar código.

3. MCP, skill y n8n

- **0:** No hay integración.
- **30:** Algún componente existe, pero no llama a la API real.
- **60:** MCP o flujo funciona parcialmente.
- **80:** Seis herramientas, skill y flujo único funcionan exitosamente con la API.
- **100:** Todo lo anterior más evidencia de cada herramienta y error controlado registrado sin crear una orden indebida.

4. Dashboard y PDF de la orden

- **0:** No hay dashboard ni demostración.
- **30:** Dashboard estático o desconectado.
- **60:** Muestra algunos datos reales, pero el flujo queda incompleto.
- **80:** Dashboard conectado; permite generar y visualizar el PDF **BORRADOR**.
- **100:** Todo lo anterior más PDF guardado con marca de agua diagonal **BORRADOR** y actualización visible tras la recepción.

5. SDD y documentación

- **0:** No hay documentación.
- **30:** Documentación mínima sin evidencia de proceso.
- **60:** Hay documentos o README, pero incompletos o inconsistentes.
- **80:** README, Swagger, diagrama y documentos SDD completos.
- **100:** Todo lo anterior más trazabilidad regla → prueba, hashes de commits y evidencias reproducibles.